Idempotent Processing

doing it twice has the same effect as once

Phase 4 videos on YouTube

In one line: messages and retries arrive more than once, so make every handler idempotent: record processed message IDs in the same transaction as the side effect, use upserts and absolute updates, and pass idempotency keys to external APIs, so a duplicate changes nothing.

1 Charged twice

A payment consumer reads a message: charge the customer $40. It charges the card, then crashes before acknowledging the message. The broker redelivers it. The consumer charges again. The customer pays $80. Nothing malfunctioned: redelivery is how brokers guarantee you don't **lose** messages.

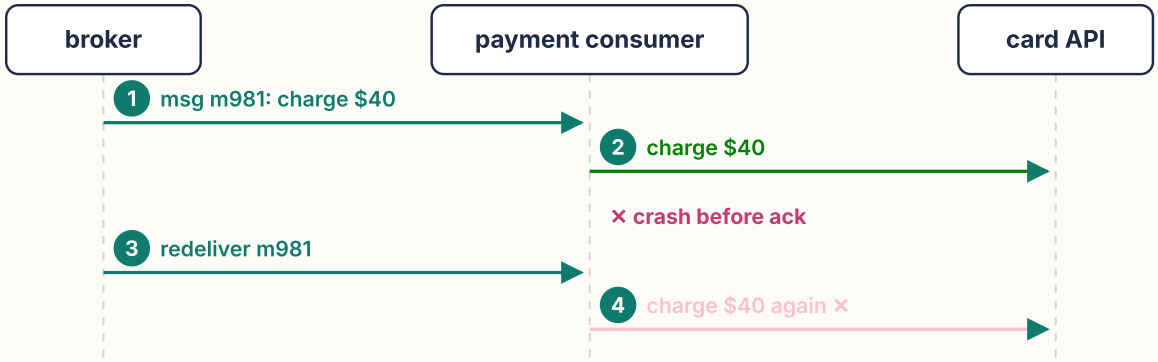


Figure 1. At-least-once delivery means a crash between the work and the ack causes a duplicate.

2 What idempotent means

An operation is **idempotent** if doing it twice has the same effect as doing it once. `SET balance = 60` is idempotent; `balance = balance − 40` is not. In distributed systems you can't prevent duplicates (retries, redeliveries, double clicks, replays), so you make processing them harmless.



Figure 2. Four ways to make a handler idempotent. Most systems combine two of them.

3 The dedup table pattern

Give every message a unique ID. In one database transaction, insert the ID into a `processed_messages` table with a unique constraint **and** apply the change. If the ID already exists, the insert fails, the transaction rolls back, and you just acknowledge the message. Because the record of processing and the effect commit together, a crash can never leave one without the other. This is also called the **inbox pattern** (lesson 38).

WORKED EXAMPLE: THE PAYMENT CONSUMER

The payment consumer receives message `m981`. In one transaction it inserts `m981` into `processed_messages`, records a pending charge, and commits. It calls the card provider with idempotency key `m981`, so if it crashes and retries, the provider returns the original charge instead of making a new one. Then it marks the order PAID and acknowledges. When the broker redelivers `m981`, the unique insert fails and the consumer just acks: one charge, however many deliveries.

```
BEGIN;
INSERT INTO processed_messages (id) VALUES ('m981');   -- unique key
--   duplicate? → unique violation → ROLLBACK, ack, skip
UPDATE orders SET status = 'PAID' WHERE id = 42;
COMMIT;
-- then acknowledge the message
```

Figure 3. Dedup and effect in one transaction. External calls get the same ID as an idempotency key.

4 Details that matter

- Use a **stable** ID created by the producer (a UUID or business key), not the broker's delivery ID, which may change on redelivery.
- Keep dedup records at least as long as duplicates can arrive (the broker's retention or retry window), then expire them.
- External side effects (emails, card charges) can't join your transaction: use the provider's idempotency keys, or a status column checked before sending.
- Prefer absolute writes (`SET`, upsert) over relative ones (`+=`).

COMMON MISTAKES

- Checking 'already processed?' in one transaction and doing the work in another.
- Dedup in an in-memory set that disappears on restart.
- Acking before doing the work, which turns duplicates into lost messages.
- Generating a new idempotency key on each retry.

5 Key terms

Idempotent: repeating it has the same effect as once

Dedup table / inbox: processed message IDs with a unique key

Upsert: insert or update by key

At-least-once: messages may be redelivered, never lost

Idempotency key: client-chosen ID the server uses to deduplicate

6 Interview questions

Q1. What is idempotent processing, and why is it needed? BASIC

Processing where duplicates have no extra effect. Brokers and retries deliver messages at least once, so without it, crashes and retries cause double effects.

Q2. How do you deduplicate messages reliably? BASIC

Store each message's unique ID in a table with a unique constraint in the same transaction as the business change; a duplicate fails the insert and is skipped.

Q3. A consumer sends welcome emails and users sometimes get two. Fix it. SCENARIO
Record 'welcome sent' per user in the database with a unique constraint before or atomically with sending, or use the email provider's idempotency key; also avoid acking before processing.

Q4. How long should you keep processed message IDs? SCENARIO
At least as long as a duplicate can still arrive: the broker's redelivery and retention window plus any replay period, then expire them with a TTL or partitioned table.

Q5. Is 'UPDATE stock = stock − 1' idempotent? How do you fix it? TRICKY
No. Record the message ID with the change, or use an absolute value with a version check, so a duplicate can't subtract twice.

Q6. Why not just use exactly-once delivery? TRICKY
End-to-end exactly-once side effects aren't possible over an unreliable network with external systems; at-least-once plus idempotency gives the same visible result.

Next: SD-H-021 · Exactly-Once Processing: Claims and Limits backendroadmap.com